1. Project Title

Concurrent Banking Transaction System

2. Group Members & Course Information

Course Title: Operating Systems (CS-2006)
Instructor: Dr. Ghufran Ahmed
Semester: Spring 2026

Group Members:

- Muhammad Zain Khan (Roll No: 24K-0838)
- Muhammad Atta Hussain (Roll No: 24K-0797)
- Muhammad Zamin (Roll No: 24K-0982)

3. Introduction / Background

Banking is one of those systems where everything must work perfectly, every single time. When someone transfers money or checks their balance, there are often hundreds of other users doing the same thing at that exact moment. Managing all of this together is not simple.

The main challenge is: how do we make sure that multiple users (or threads) accessing the same account don't create errors? For example, if two people try to withdraw money at the same time, the system must ensure the balance stays correct.

In this project, we are building a **banking transaction simulation in C++** where multiple **threads** act as clients performing deposits, withdrawals, and transfers simultaneously. To handle this safely, we will use **Operating System concepts** like **mutexes, semaphores, and thread synchronization** to ensure everything stays consistent—just like real banking systems.

4. Problem Statement

Consider a situation where two threads read the same account balance of Rs. 5,000 at the same time. Both think withdrawing Rs. 4,000 is allowed, and both complete the transaction. This results in a negative balance, which is incorrect.

This issue is called a **race condition**, and it happens when multiple threads access shared data without proper control.

Our goal is to prevent such problems by:

- Ensuring only one thread can modify an account at a time using **mutexes**
- Avoiding **deadlocks** during transfers between accounts
- Managing high numbers of requests using **semaphores**

These are real-world problems in financial systems, and this project helps us understand how they are handled.

5. Objectives of the Project

- Develop a **multi-threaded banking simulation** in C/C++
- Use **mutexes** to protect account balances (critical sections)
- Use **semaphores** to limit concurrent transactions
- Ensure balances never go negative and total money remains consistent
- Maintain a **transaction log with timestamps**
- Handle transfers safely without causing **deadlocks**
- Display system activity through a GUI or clean console output

6. Scope of the Project

This project focuses only on the **synchronization and concurrency aspects** of banking systems.

What we will include:

- Concurrent deposits, withdrawals, and transfers using **multiple threads**
- **Semaphore-based control** for limiting active transactions
- A complete **transaction history** for verification
- Extra features like **VIP priority transactions** and basic **fraud detection**

What we will NOT include:

- No real banking system, database, or network integration
- The system will run entirely in memory

This keeps our focus on **Operating System concepts** rather than full application development.

7. Literature Review / Related Work

This project is closely related to classic OS problems like:

- **Producer-Consumer Problem**
- **Readers-Writers Problem**

These problems deal with managing shared resources using **mutexes and semaphores**, which is exactly what we are applying here.

From a database perspective, this relates to **ACID properties**:

- **Consistency** and **Isolation** are especially important in our system

One major issue we studied is **deadlock**. For example:

- Thread A locks Account 1 and waits for Account 2
- Thread B locks Account 2 and waits for Account 1

Both threads get stuck forever.

To prevent this, we will use **consistent lock ordering** (e.g., always lock accounts based on ID).

8. Proposed System / Methodology

We will build the system step by step:

1. Start with a simple account system (single-threaded)
2. Introduce **multithreading** and apply **mutexes** to fix race conditions
3. Add **semaphores** and implement transaction logging
4. Add advanced features like:
    - VIP priority handling
    - Rollback for failed transactions
    - Fraud detection
5. Finally, improve output (GUI or console), testing, and documentation

9. System Architecture / Design

The system will have the following modules:

- **Account Manager**
  Manages all account data and ensures safe access using mutexes
- **Transaction Threads**
  Each thread represents a client performing operations concurrently
- **Mutex Layer**
  Each account has its own mutex to protect its balance
- **Semaphore Controller**
  Limits how many transactions run at the same time
- **Transaction Logger**
  Keeps a record of all operations with timestamps
- **Consistency Verifier**
  Checks that total system balance remains correct

This modular design makes the system easier to build, test, and debug.

10. Operating System Concepts Used

- **Multithreading (pthreads)** — multiple clients running simultaneously
- **Mutexes (Mutual Exclusion)** — protecting shared data
- **Semaphores** — controlling concurrency
- **Critical Sections** — safe access to account balances
- **Deadlock Prevention** — avoiding circular waiting
- **Race Condition Handling** — ensuring correct results
- **Thread Synchronization** — coordinating execution

11. Tools and Technologies

- **Programming Language:** C/C++
- **Environment:** GCC (Linux) / VS Code
- **Threading Library:** POSIX Threads (pthreads)
- **Version Control:** GitHub
- **GUI (optional):** Qt or GTK
- **Fallback:** Console-based output

12. Implementation Plan & Timeline (Tentative)

Week	Task
Week 9	Account structure design and basic single-threaded operations
Week 10	Multi-threaded implementation with mutex-protected account access
Week 11	Semaphore integration and concurrent transaction limiting
Week 12	Transaction logging and balance consistency verification
Week 13	Additional features: VIP priority, fraud detection, rollback
Week 14	GUI development and system integration
Week 15	Final testing, documentation, and presentation

13. Expected Results and Conclusion

By the end of this project, we expect to have a working banking simulator that handles multiple concurrent clients without any data corruption, deadlocks, or incorrect balances. If two threads try to touch the same account at the same time, one will wait. If a transfer tries to grab locks in the wrong order, it will be prevented. And throughout all of this, every transaction will be logged and the total money in the system will always add up correctly.

More than just fulfilling course requirements, we think this project teaches something genuinely useful. Concurrency bugs are notoriously hard to reproduce and debug in real systems — building one from scratch and deliberately fixing these issues is probably the best way to truly understand them. We are also hoping this ends up being something worth showing to employers, since thread safety and concurrent programming come up in almost every system or backend role.